

Predicting User Sentiment in Software Issue Reports

ST2MLE — Machine Learning for IT Engineers

A pipeline to classify issue-report sentiment: negative · neutral · positive

Comparing vectorisation strategies and classifiers, with full tuning, imbalance handling, dimensionality reduction, pruning and early stopping.

Problem & objectives

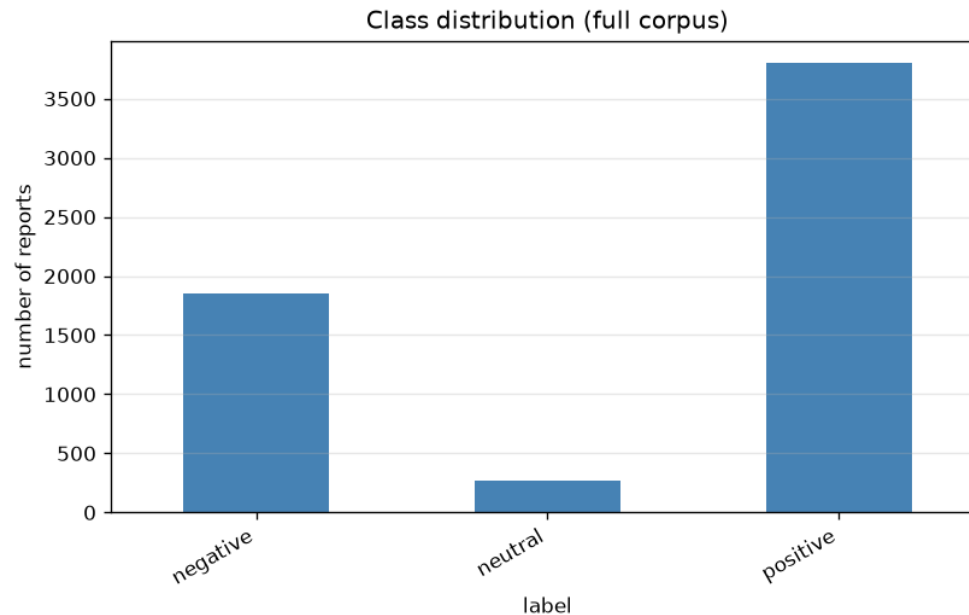
- **Goal:** automatically classify the sentiment of user-reported software issues (bug reports, feature requests, reviews) into **negative / neutral / positive**.
- **Why it matters:** triage angry reports faster, spot satisfaction trends, prioritise fixes.
- **What we compare:**
- 4 vectorisers — Bag-of-Words, TF-IDF, Word2Vec (in-domain), pre-trained Word2Vec (+ BERT code)
- 5 classifiers — Naive Bayes, Decision Tree, Random Forest, AdaBoost, Logistic Regression
- **And we study:** class imbalance (SMOTE), PCA, tree pruning, early stopping.

Dataset — real Facebook reviews

- **Real Facebook app reviews** (Kaggle): 10 k reviews, **content** + 1-5 **score**.
- **Stars → sentiment**: 1-2★ negative · 3★ neutral · 4-5★ positive.
- De-duplicated (no train/test leakage) → **5 923 unique reviews**, natural skew:

negative	neutral	positive
~31 %	~5 %	~64 %

- **Genuinely hard**: proxy labels, short/multilingual text, only 4.5 % neutral. (Google Play & synthetic sources also wired in.)



Pipeline overview

raw text → preprocess → vectorise → [balance] → [reduce] → classify → evaluate

- **Preprocess:** lower-case, strip punctuation/URLs, tokenise, drop stop-words (**keep negations**), WordNet lemmatise.
- **Vectorise:** BoW · TF-IDF · Word2Vec · pre-trained Word2Vec (Google-News).
- **Balance (train only):** SMOTE / under-sampling via `imblearn`.
- **Reduce:** TruncatedSVD (PCA for sparse text).
- **Classify & tune:** 5 models, 5-fold CV, `GridSearchCV`.
- **Evaluate:** accuracy, macro P/R/F1, confusion matrix.

Engineering: preprocess once + cache vectorisers → whole study in ~8 min.

Preprocessing

- **Clean:** lower-case → remove URLs/HTML/punctuation/digits.
- **Tokenise** then **remove stop-words** — but **keep negations** (not , no , never ...) because they flip sentiment.
- **Lemmatise** (WordNet, verb-then-noun): crashing → crash, issues → issue.
- Implemented as a stateless scikit-learn transformer → no train/test leakage.

Example "Critical issue -- The app keeps crashing!!!" → critical issue app keep crash

Vectorisation — 4 strategies

Strategy	Idea	Output
Bag-of-Words	n-gram counts	sparse, ≥ 0
TF-IDF	counts $\times$ inverse document frequency	sparse, ≥ 0
Word2Vec	mean of embeddings trained on our corpus	dense
Word2Vec-pretrained	mean of Google-News 300-d vectors (the brief's "pre-trained Word2Vec")	dense

- Sparse + non-negative $\rightarrow$ feeds **MultinomialNB**; dense $\rightarrow$ **GaussianNB**.
- **BERT** vectoriser is implemented & documented; needs the model hub (unavailable here) so it is skipped — a Colab-ready script adds it elsewhere.

Models & tuning

- **Naive Bayes** — `MultinomialNB` (sparse) / `GaussianNB` (dense), auto-selected.
- **Decision Tree** — pre-pruning (`max_depth`, `min_samples_leaf`) **and** post-pruning (`ccp_alpha`).
- **Random Forest** — bagged trees.
- **AdaBoost** — boosted stumps.
- **Logistic Regression** — demonstrates **L2 regularisation** (tuned `C`).

All tuned with `GridSearchCV`, **5-fold cross-validation**, scoring **macro-F1**.

Experimental setup

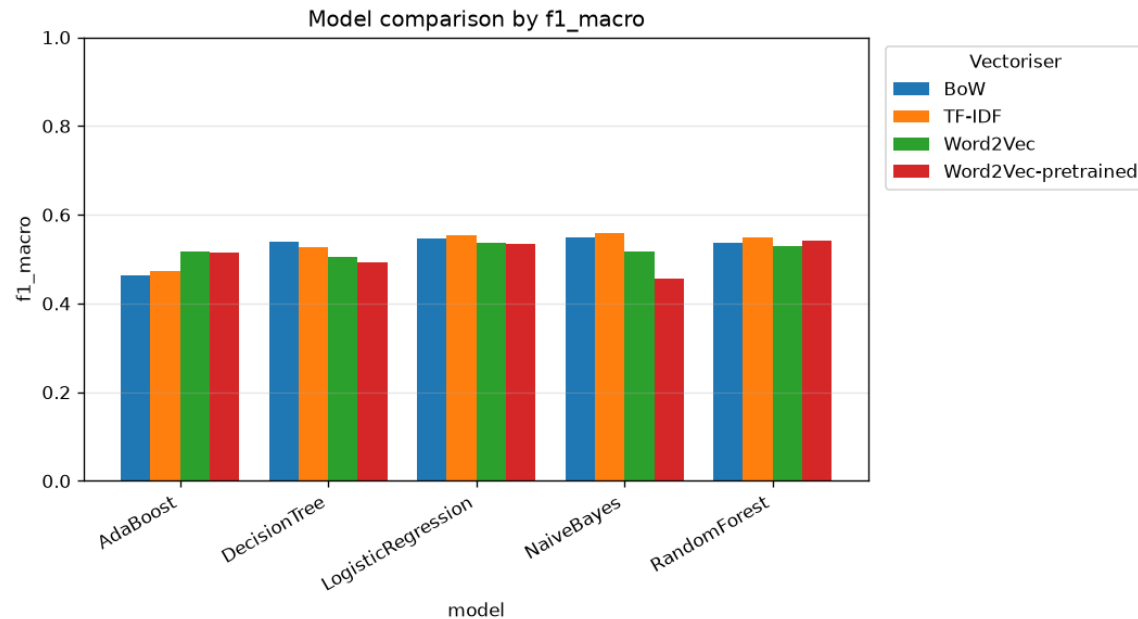
- **Split:** stratified 80 % train / 20 % test.
- **CV:** 5-fold `StratifiedKFold` on the training set.
- **Tuning target:** macro-F1 (weights every class equally on imbalanced data).
- **Reproducibility:** single seed (42); deterministic — a clean re-run reproduces every number byte-for-byte.
- **Compute:** 4 cores; full study $\approx$ 8 min thanks to cached vectorisers.

Results — model comparison

Best classifier per vectoriser (test macro-F1):

Vectoriser	Best classifier	Macro-F1
TF-IDF	Naive Bayes	0.559
BoW	Naive Bayes	0.548
Word2Vec-pretrained (Google-News)	Random Forest	0.542
Word2Vec (in-domain)	Logistic Regression	0.537

Macro-F1 $\approx$ 0.46–0.56 (real, noisy data); top 5 are all TF-IDF/BoW. AdaBoost worst. Full 20-row table in the report.



Results — best model

TF-IDF + Naive Bayes — macro-F1 0.559, accuracy 0.839 — confusion matrix on the held-out test set:

TF-IDF + NaiveBayes

	negative	neutral	positive
True label negative	291	0	81
True label neutral	22	0	31
True label positive	57	0	703
	negative	neutral	positive

Predicted label

- High accuracy is driven by the dominant **positive** class.
- The tiny **neutral** (3★, 4.5 %) class is hardest — it overlaps both neighbours.

Class-imbalance handling

TF-IDF + Logistic Regression; only the resampling strategy changes (train folds only).

Strategy	Accuracy	Macro-F1	Recall neu
none	0.836	0.555	0.000
SMOTE	0.727	0.551	0.189
under-sampling	0.601	0.505	0.472

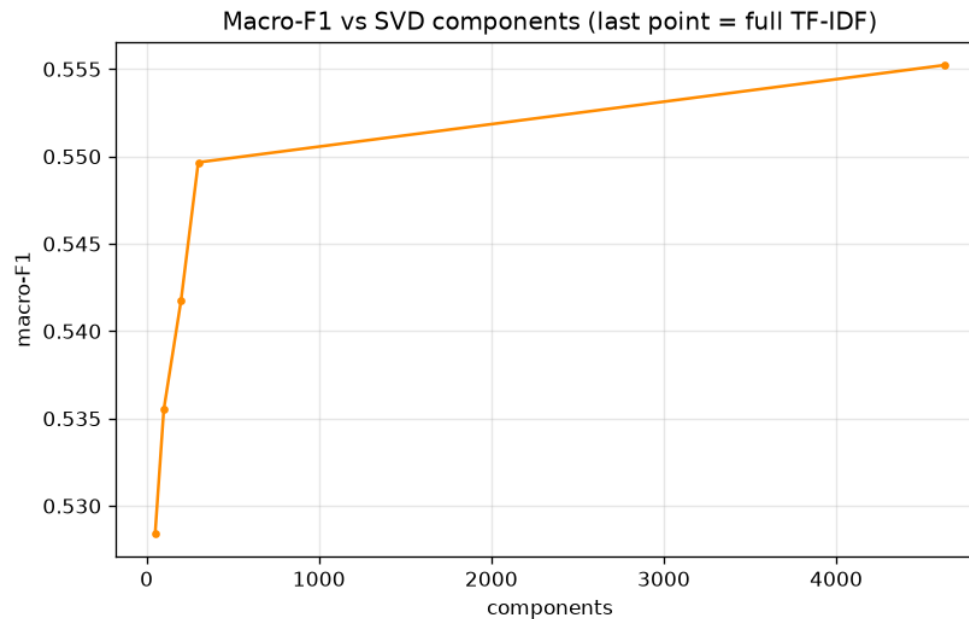
Takeaway — the most striking result: the baseline hits 0.84 accuracy while **never predicting a single neutral** (recall **0.000**). SMOTE/under-sampling recover neutral recall (→0.19 / →0.47) → **accuracy is a trap; resampling rescues the minority class.**

Dimensionality reduction (PCA / TruncatedSVD)

TF-IDF (4 630 features) → TruncatedSVD; downstream macro-F1 (Logistic Regression):

Components	Variance	Macro-F1
100	37 %	0.536
300	56 %	0.550
4 630 (full)	100 %	0.555

Takeaway: 300 components (6.5 % of features) keep ~99 % of full performance — a ~15× compression, and essential before dense models (e.g. boosting).

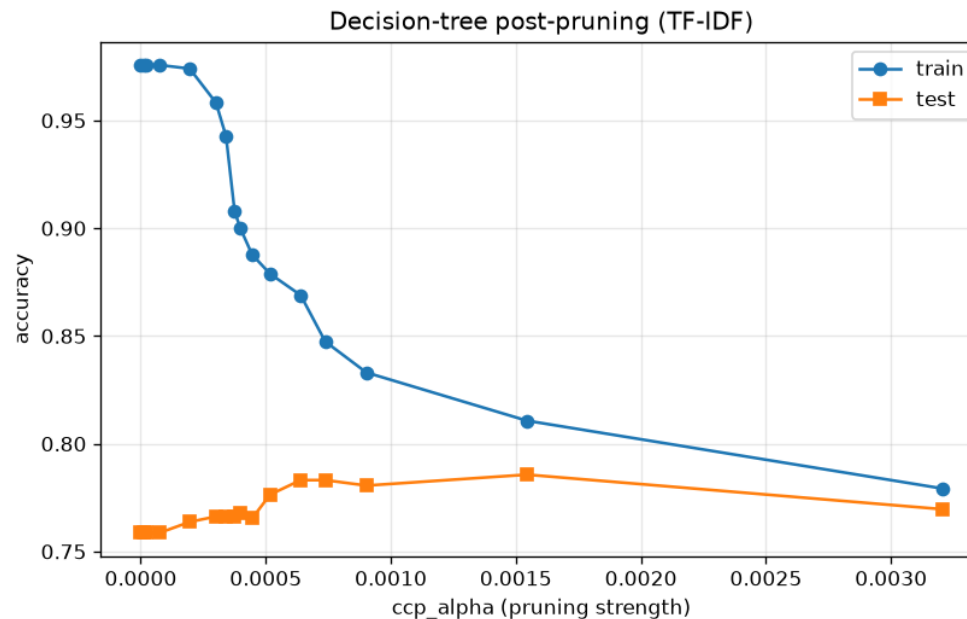


Decision-tree pruning

Cost-complexity (`ccp_alpha`) post-pruning on TF-IDF:

Tree	# nodes	Train acc	Test acc
unpruned	1 857	0.976	0.759
best pruned	69	0.811	0.786
over-pruned	29	0.779	0.770

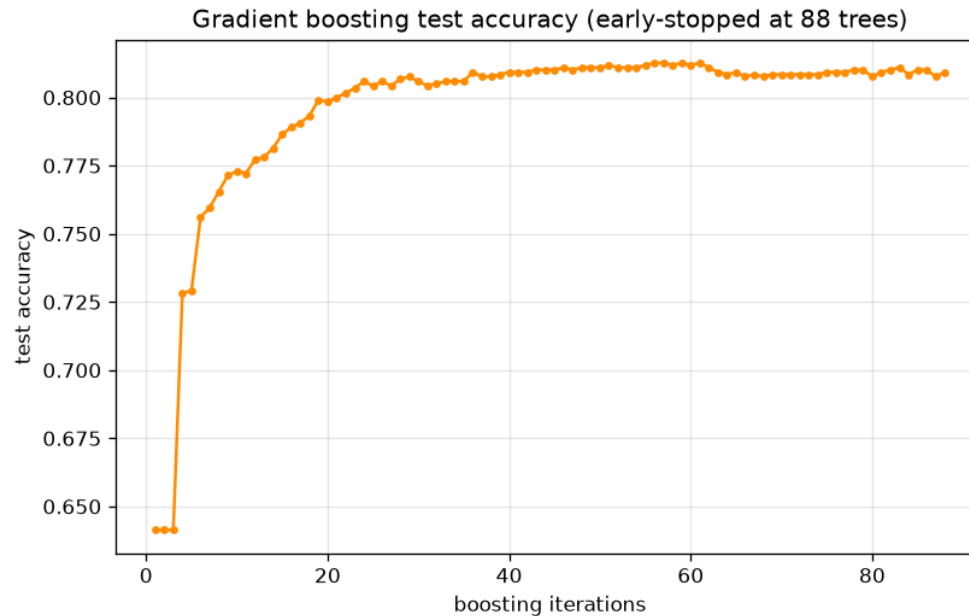
Takeaway: pruning cuts 1 857 → 69 nodes, **raises test accuracy 0.76 → 0.79**, and shrinks the over-fitting gap 0.22 → 0.03 — textbook bias-variance.



Early stopping (gradient boosting)

TF-IDF → SVD(100) → Gradient Boosting, ceiling 500 trees, `n_iter_no_change=10`.

- **Stopped after 88 of 500 trees** — ~6× less training, no loss of test quality.
- Validation monitoring halts once added trees stop helping → guards over-fitting.



Discussion — trade-offs

- **Simple sparse models win** on short, noisy reviews — TF-IDF + Naive Bayes (0.559); the top 7 are all TF-IDF/BoW.
- **Accuracy is a trap** at 64 % positive (baseline never predicts neutral); **macro-F1** is the metric that matters.
- **AdaBoost** is worst (stumps see 1 word of thousands); **embeddings** (Word2Vec, pre-trained Word2Vec) sit mid-table — mean-pooling washes out lexical cues.
- **SMOTE** rescues minority recall; **PCA** buys efficiency; **pruning & early stopping** both curb over-fitting.

Challenges & future work

Challenges - Star ratings are an imperfect (proxy) sentiment label; heavy de-duplication + multilingual noise shrink 10 k raw reviews to 5.9 k usable ones. - Severe imbalance (4.5 % neutral) → low macro-F1 despite high accuracy. - MultinomialNB rejects negatives → auto-switch to GaussianNB for embeddings.

Future work - Add GitHub/JIRA issues directly once API access is available. - Enable BERT / fine-tuned transformers (code already provided). - Class-weighting and cost-sensitive thresholds as further imbalance remedies.

Conclusion

- Built a **complete, reproducible** sentiment pipeline on **real Facebook reviews**.
- Compared **4 vectorisers × 5 classifiers**, fully tuned with 5-fold CV.
- **Best: TF-IDF + Naive Bayes (macro-F1 0.559)**; SMOTE was decisive for the minority.
- Demonstrated imbalance handling, PCA, pruning and early stopping end to end — with an honest account of why real-data scores are modest.

Thank you — questions?